

Performance automation framework designed using ICEPOT framework and TLDR technique.

Prompt:

Context Act as an performance test architect to generate the performance test agentic AI framework using JMeter and K6 tools. I

nstruction:

[ASK] Ask me 10 multiple-choice questions with radio selection, one after another. Like, once the first question is answered, the second question should be asked.

[MANDATORY] The framework should get proceeded with step by step only once all 10 questions get answered.

[NOTE] framework's next step will start once the first step of framework creation is completed and the user acknowledges the previous steps as done.

[SUGGESTION] Framework should be Python-based and should use an open-source LLM for no cost. It should be an MD file to generate a framework.

[MANDATORY] It should handle the steps—entire logic from script creation based on the user flow, auto-correlation handler, script execution, report analysis, and anomaly detection using LLM.

[CAUTION] It should be executed on a local machine, i.e., downloadable and CICD-friendly. Also, it should be able to execute for any server, any environment, and any CSV data set; the user can provide input from his end, based on which environment he wants to execute, and he can provide a CSV of dynamic user data for execution.

[MANDATORY] While creating the script, it should not record any other request, excluding domain requests only.

Example:

If the user is saying to execute the script for login flow Users: 20 Ramp up: 5 duration: 30 Loop: forever Think time: 5s Pacing: 2s csv file path Flow: 1. User goto <https://load1.gy.com> url and load url 2. User loginw ith user majordomo/sa 3. Once logged in, user navigate on dashboard.

Persona: Act as a performance expert to review the created , designed agentic performance framework.

Output: Framework should be scalable and access by anyone and anywhere

Tone: Soft and professional

Question 1 of 10 — Primary Execution Engine Preference

Which execution strategy do you want the Agentic Performance Framework to prioritize?

- A. Apache JMeter only
- B. k6 only
- C. Hybrid approach using both Apache JMeter and k6
- D. Auto-select engine based on protocol/application type
- E. Modular plugin-based execution engine (future extensibility)

Please reply with only the option letter (A/B/C/D/E).

D,E

Question 2 of 10 — Preferred AI Model Strategy

Which open-source LLM approach should the framework use for script generation, correlation handling, anomaly detection, and report summarization?

- A. Local lightweight model using Ollama + Llama 3
- B. Local coding-focused model using DeepSeek Coder
- C. Multi-model routing (use different models for scripting, RCA, and analytics)
- D. GPU-optimized local deployment using vLLM
- E. CPU-only optimized deployment for normal laptops/desktops

Please reply with the option letter(s) only.

A,C

Question 3 of 10 — Script Creation Input Method

How should the framework generate performance scripts?

- A. Plain English/Natural language prompts
- B. HAR file import and conversion
- C. Browser recording + cleanup automation
- D. Swagger/OpenAPI/API collection import
- E. Hybrid mode supporting all inputs

Please reply with the option letter(s) only.

C,E

Question 4 of 10 — Auto-Correlation Capability Scope

Which correlation mechanisms should the framework automatically handle?

- A. Session IDs and cookies
- B. CSRF/authentication tokens
- C. Dynamic hidden parameters and headers
- D. GraphQL/WebSocket dynamic values
- E. AI-based intelligent correlation for unknown patterns

Please reply with the option letter(s) only.

A,B,C,E

Question 5 of 10 — Preferred CI/CD & Execution Integration

Where should the framework be able to execute automatically?

- A. Local machine standalone execution
- B. Jenkins integration
- C. GitHub Actions integration
- D. GitLab CI/CD integration
- E. Docker containerized execution

Please reply with the option letter(s) only.

c

Question 6 of 10 — Reporting & Observability Stack

Which reporting and monitoring capabilities should the framework include?

- A. HTML performance reports
- B. Real-time dashboards using Grafana
- C. Metrics storage using InfluxDB
- D. AI-generated Root Cause Analysis (RCA) summaries
- E. Trend comparison across multiple executions

Please reply with the option letter(s) only.

A,B,D,E

Question 7 of 10 — Target Application/Protocol Coverage

Which application types should the framework support?

- A. Web applications (HTTP/HTTPS)
- B. REST/SOAP APIs
- C. Mobile backend APIs
- D. WebSocket/Streaming applications
- E. Database and backend service performance validation

Please reply with the option letter(s) only.

A,B,C,E

Question 8 of 10 — Intelligent Anomaly Detection Expectations

What type of AI-driven anomaly detection should the framework perform?

- A. Response time spike detection
- B. Error pattern clustering
- C. Infra bottleneck correlation (CPU, memory, DB)
- D. Baseline vs current execution deviation analysis
- E. Predictive failure/risk forecasting

Please reply with the option letter(s) only.

A,B,C,D,E

Question 9 of 10 — Framework Architecture Preference

Which architectural model should the agentic framework follow?

- A. Monolithic Python application
- B. Microservices-based modular agents
- C. Event-driven orchestration architecture
- D. Plugin-based extensible framework
- E. Hybrid architecture combining modular agents + plugins

Please reply with the option letter(s) only.

B

Question 10 of 10 — User Interaction & Execution Experience

How should users interact with the framework?

- A. CLI-based execution only
- B. Web UI dashboard
- C. YAML/JSON configuration-driven execution
- D. Conversational AI assistant interface
- E. Hybrid approach (CLI + Config + AI assistant + UI)

Please reply with the option letter(s) only.

B

All 10 questions are now completed successfully.

Your Selected Architecture Summary

Area	Selected Direction
Execution Engine	Auto-select + extensible plugin engine
LLM Strategy	Ollama + Llama 3 with multi-model routing

Area	Selected Direction
Script Generation	Browser recording + hybrid ingestion
Correlation	AI-based intelligent correlation
CI/CD	GitHub Actions
Reporting	HTML + Grafana + AI RCA
Protocols	Web, API, Mobile APIs, DB
AI Analytics	Full anomaly + prediction suite
Architecture	Microservices-based
UX	Web dashboard

Framework Creation — Step 1

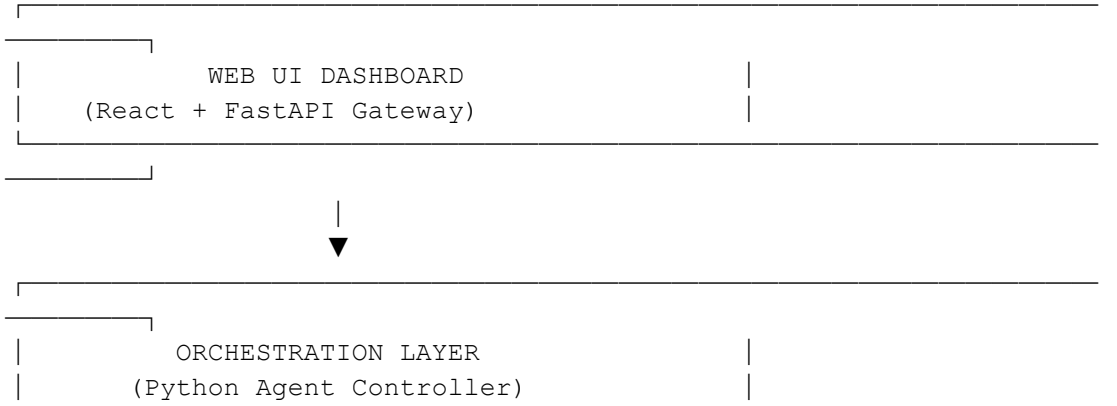
Step 1 Objective

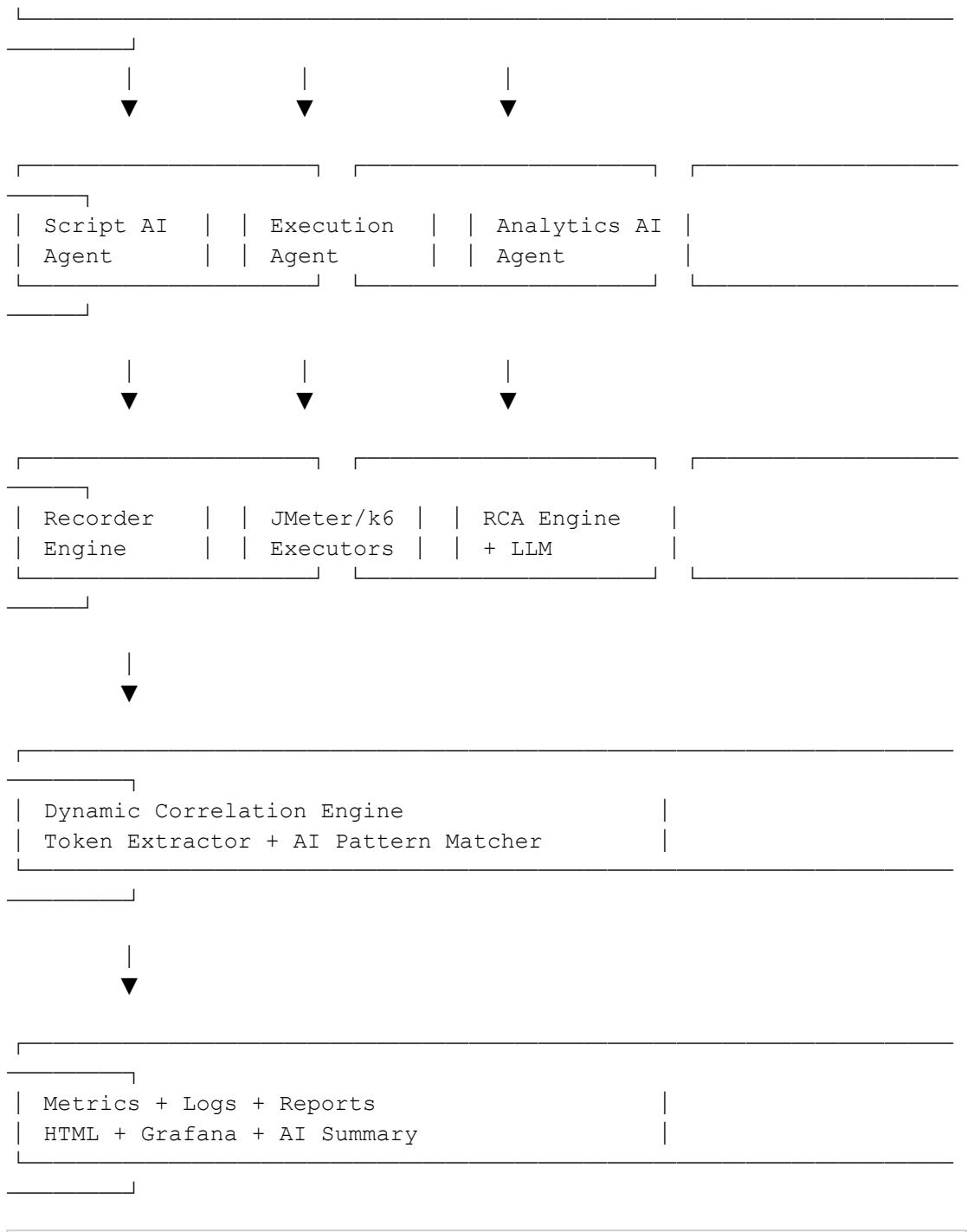
Design the **High-Level Architecture Blueprint** for the Agentic AI Performance Testing Framework.

As requested, the framework will:

- Be Python-based
- Run locally
- Be fully open-source
- Support both Apache JMeter and k6
- Support AI-based script generation and anomaly detection
- Be CI/CD friendly
- Support any environment and CSV datasets
- Filter only domain-specific requests during recording
- Be scalable and modular

High-Level Proposed Architecture





Core Technology Stack

Layer	Technology
Backend	Python + FastAPI
Frontend	React
AI Runtime	Ollama
LLM	Llama 3

Layer	Technology
Execution Engines	Apache JMeter + k6
Monitoring	Grafana
Storage	SQLite/PostgreSQL
Messaging	Redis/RabbitMQ
CI/CD	GitHub Actions
Packaging	Docker
Recording	Playwright + BrowserMob Proxy

Key Enterprise Features

1. AI-Based Script Generator

Input:

- Natural language
- HAR recording
- Browser recording
- Flow definition

Output:

- Auto-generated JMeter/K6 scripts
 - Parameterization
 - Correlation handling
 - Assertions
 - CSV integration
-

2. Intelligent Request Filtering

The recorder agent will:

- Capture only target-domain requests
- Ignore CDN/static/analytics calls
- Remove images/fonts/unwanted third-party traffic

Example:

```
allowed_domains:  
- load1.gy.com
```

```
blocked_extensions:  
- png  
- jpg
```

- jpeg
 - gif
 - css
 - svg
 - woff
-

3. AI Auto-Correlation Engine

Handles:

- Session IDs
- CSRF tokens
- Auth headers
- Dynamic cookies
- Hidden form fields
- Unknown dynamic patterns

Mechanism:

```
Traffic Capture
      ↓
Dynamic Value Detection
      ↓
LLM Pattern Classification
      ↓
Extractor Generation
      ↓
Auto Script Injection
```

4. Execution Layer

Supports:

- Local execution
- Distributed execution
- Multi-environment execution
- CSV-driven dynamic data
- Environment profiles

Example:

```
environment: QA

users: 20
rampup: 5
duration: 30m
loop: forever
think_time: 5s
pacing: 2s

csv_path: ./data/users.csv
```

5. AI Analytics Engine

Performs:

- Response time anomaly detection
- Error clustering
- SLA violation analysis
- Trend comparison
- Root cause suggestions
- Predictive risk detection

Example Output:

High latency detected in Login API.

Potential reasons:

- DB connection saturation
- Thread pool exhaustion
- Redis timeout spike

Confidence Score: 89%

Proposed Repository Structure

```
agentic-performance-framework/  
├── ui/  
├── gateway/  
├── agents/  
│   ├── recorder-agent/  
│   ├── script-agent/  
│   ├── execution-agent/  
│   ├── analytics-agent/  
│   └── correlation-agent/  
├── engines/  
│   ├── jmeter/  
│   └── k6/  
├── datasets/  
├── reports/  
├── configs/  
├── templates/  
├── docker/  
├── ci/  
└── docs/
```

Scalability Strategy

Capability	Strategy
Horizontal scaling	Microservice agents
Future tool support	Plugin architecture
Multi-user execution	Queue-based orchestration
Multi-environment	Dynamic config loader
Large-scale execution	Distributed runners

Security & Governance

- Environment variable secret handling
 - Masking sensitive credentials
 - Role-based UI access
 - Execution audit logging
 - Secure CSV encryption support
-

Step 1 Deliverables Completed

Architecture blueprint
Technology selection
AI strategy definition
Correlation design
Scalability model
Execution strategy
Reporting strategy

Please review Step 1 carefully.

Once you acknowledge with:

- “Done”
or
- “Proceed”

I will start **Step 2 — Detailed Microservice & Agent Design**, where we will define:

- Each agent’s responsibilities
- Internal workflows
- Prompt engineering strategy
- LLM orchestration
- Correlation algorithms

- JMeter/K6 generation pipeline
- Data flow contracts
- API design
- Execution lifecycle
- Failure handling architecture
- Plugin extensibility model

TL;DR Framework designed steps

TL;DR — Agentic AI Performance Testing Framework Design Steps

Step 1 — Architecture Blueprint

Designed:

- Python-based microservices architecture
 - Web UI + AI agents
 - Apache JMeter + k6 execution engines
 - Local execution + CI/CD compatibility
 - AI-driven script generation, correlation, execution, and analytics
-

Step 2 — Agent & Microservice Design

Will define:

- Recorder Agent
 - Script Generator Agent
 - Correlation Agent
 - Execution Agent
 - Analytics/RCA Agent
 - Agent communication workflows
 - API contracts and orchestration logic
-

Step 3 — Recording & Smart Traffic Filtering

Will implement:

- Browser recording
- Domain-only request filtering
- Ignore CDN/static/analytics requests
- HAR cleanup automation
- User flow extraction

Step 4 — AI Script Generation Engine

Will generate:

- JMeter/K6 scripts automatically
- CSV parameterization
- Assertions
- Think time/pacing
- Multi-environment support
- Dynamic execution configs

Step 5 — AI Auto-Correlation Engine

Will handle:

- Session IDs
- CSRF tokens
- Dynamic cookies/headers
- Hidden parameters
- Unknown dynamic patterns using LLM detection

Step 6 — Execution Orchestration Layer

Will support:

- Local/distributed execution
- Environment switching
- CSV-driven datasets
- Scheduling
- Parallel execution
- CLI/UI/API-based execution

Step 7 — Monitoring & Reporting

Will provide:

- HTML reports
- Grafana dashboards
- SLA tracking
- Historical trends
- Centralized metrics/logging

Step 8 — AI Analytics & Anomaly Detection

Will perform:

- Response time anomaly detection
- Error clustering
- RCA suggestions
- Infra correlation
- Predictive failure analysis

Step 9 — CI/CD & Packaging

Will include:

- GitHub Actions integration
- Docker packaging
- One-click execution
- Portable deployment
- Environment-based pipelines

Step 10 — Documentation & MD Framework Generation

Final deliverables:

- Complete `.md` framework documentation
- Setup guide
- API guide
- Architecture diagrams
- Execution examples
- Extensibility documentation
- Open-source ready repository structure